

Project 4: Pager – A Virtual Memory Manager

Nolan Tuttle & Mathew Hobson

College of Engineering and Technology, Grand Canyon University

CST-315: Operating Systems Lecture and Lab

Professor Citro

March 23, 2025

Summary

This program simulates the process that an operating system uses to implement virtual memory paging. This includes implementation of a page table, address translation, virtual (logical) and physical memory in the form of input and output text files respectively. First, the program takes an input text file and assigns it to a virtual memory array. Then it reads 4 bytes for each page, assigning each 4-byte page its own unique spot in the page table corresponding to the physical memory output. Then, each page in the table is iterated through, assigning each 4-byte page to its respective frame in physical memory using the page table. It prints when complete and outputs a text file output.txt.

Code Execution

```
(base) Nolans-MacBook-Air:Project 4 Pager - A Virtual Memory Manager nolan$ ./pager
Logical memory has been stored.
Physical memory written to output.txt
All pages have successfully been mapped. Exiting the program.
```

Approach/Methodology

In actual paging in operating systems, there are a few different approaches that differ slightly from this straightforward iteration used in this program. For example, demand paging is a common mechanism used that only does address translation, that is, moving logical to physical memory, when the process running requires it (Stallings, 2019). Otherwise, it remains in logical memory, that way CPU power is not wasted always translating parts from logical to physical memory (Stallings, 2019).

References

Stallings, W. (2019). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.

Tuttle, N., Hobson, M., (2025) *CST315* GitHub. <https://github.com/nolantuttle/CST315.git>